

Министерство образования Республики Беларусь

Учреждение образования

Белорусский государственный университет информатики и  
радиоэлектроники

Кафедра электронных вычислительных машин

## **ПОЯСНИТЕЛЬНАЯ ЗАПИСКА**

к курсовому проекту на тему

«Разработка микро-ЭВМ на ПЛИС»

Студент: Власов

Руководитель: \_\_\_\_\_

Минск 2026

## СОДЕРЖАНИЕ

Введение

1 Разработка общей структуры микро-ЭВМ

2 Разработка основных устройств микро-ЭВМ

3 Функциональное моделирование

4 Анализ и оптимизация разработанной микро-ЭВМ

Заключение

Список использованных источников

Приложения

## ВВЕДЕНИЕ

Целью курсового проекта является разработка микро-ЭВМ на ПЛИС в соответствии с индивидуальным техническим заданием по дисциплине «Структурная и функциональная организация ЭВМ». Введение фиксирует основные исходные параметры системы, от которых далее зависят структура команд, состав функциональных блоков, набор управляющих сигналов и перечень временных диаграмм.

Разрабатываемая микро-ЭВМ имеет следующие основные магистрали:

- шина данных разрядностью 16 бит;
- адресная шина разрядностью 16 бит;
- управляющая шина, разрядность которой определяется набором сигналов чтения и записи памяти, разрешения регистров, выбора источников внутренней шины, управления АЛУ, стеком, кэшем, КПДП, арбитром и предсказателем переходов.

Архитектура микро-ЭВМ принята гарвардской: ПЗУ команд и ОЗУ данных имеют отдельные тракты обращения. Это позволяет выборке команды не конфликтовать с обычным доступом к данным, однако обращения процессора и КПДП к ОЗУ всё равно требуют арбитража. Команды имеют фиксированный двухсловный формат: первое слово содержит код операции и номер регистра, второе слово используется как прямой адрес операнда, адрес перехода или резервное поле.

Тип адресации в проекте прямо-регистровый. Операнд или приёмник результата задаётся номером одного из 12 регистров общего назначения R0-R11, а адрес ячейки памяти располагается во втором слове команды. Для команд JMP и JZ второе слово содержит адрес следующей команды в пространстве ПЗУ.

Арифметико-логическое устройство должно выполнять операции INCS, OR, NOR и SRA. Операция INCS использует значение флага S, операции OR и NOR выполняют побитовую логическую обработку, а SRA реализует арифметический сдвиг вправо с сохранением знакового бита. Результаты операций сопровождаются формированием флагов Z, S, C и O.

Кэш-память данных организуется как четырёхканальная множественно-ассоциативная структура с вытеснением строки по наибольшей давности хранения. Синхронизация с основной памятью выполняется по схеме

сквозной записи: при записи слово согласуется и с кэшем, и с ОЗУ, поэтому последующее чтение получает актуальное значение.

Арбитр шин реализуется как централизованный параллельный блок. Его назначение - исключить одновременное управление общей магистралью данных со стороны процессорной части и контроллера прямого доступа к памяти. Блок предсказания переходов выполняется отдельно от устройства управления и использует схему A4 с индексированием по сочетанию младших разрядов счётчика команд и регистра глобальной истории GHR.

Стек реализуется отдельным блоком глубиной 7 слов по 16 бит с направлением роста вниз. Контроллер прямого доступа к памяти передаёт в ОЗУ блок объёмом 6 байт, что при словной организации памяти соответствует трём 16-разрядным словам, начиная с адреса 000Ah.

Разрабатываемая микро-ЭВМ является многотактовой, а не конвейерной. Поэтому выполнение команды рассматривается как последовательность фаз выборки, декодирования, обращения к операндам, исполнения и записи результата. Такой подход упрощает проверку временных диаграмм и позволяет отдельно показать работу ПЗУ, ОЗУ, АЛУ, стека, кэша, КПДП, арбитра и предсказателя переходов.

Разработка и проверка проекта выполняются в среде Altera Quartus II 9.1 с использованием VHDL-описаний и функционального моделирования. Итогом работы должна стать согласованная структура микро-ЭВМ, набор приложений с графическими схемами и листингами, а также результаты тестирования, подтверждающие корректность работы отдельных блоков и всей системы.

## **1 РАЗРАБОТКА ОБЩЕЙ СТРУКТУРЫ МИКРО-ЭВМ**

В данном разделе определяется архитектурная основа проектируемой системы, фиксируется набор её функциональных блоков, разрабатывается система команд и описывается взаимодействие всех узлов при исполнении программы. Именно этот раздел задаёт ту логическую модель, по которой далее будут проектироваться отдельные схемы, временные диаграммы и результаты моделирования.

### **1.1 Функциональный состав микро-ЭВМ**

**Корневыми блоками микро-ЭВМ являются:**

- блок запоминающих устройств:
  - ПЗУ программ для хранения двухсловных команд;
  - ОЗУ данных для хранения операндов, результатов и области обмена с КППД;
  - кэш-память данных со схемой управления, тегами, признаками действительности и счётчиками давности;
  - схема согласования обращений к памяти через арбитр шин.
- блок устройства управления:
  - регистр текущей команды IR0/IR1;
  - декодер кода операции и номера регистра;
  - фазовый автомат выполнения команды;
  - схема выбора следующего значения указателя команд IP;
  - формирователь управляющих сигналов для РОН, АЛУ, стека, кэша, ОЗУ, КППД и предсказателя переходов.
- блок регистров общего назначения:
  - 12 регистров R0-R11 разрядностью 16 бит;
  - два порта чтения и один порт записи для выдачи операндов и фиксации результата.
- блок арифметико-логического устройства:
  - узлы операций OR и NOR;
  - узел арифметического сдвига вправо SRA;
  - узел инкремента по флагу S для операции INCS;
  - схема формирования флагов Z, S, C и O.
- блок стека:
  - стековая память глубиной 7 слов;
  - указатель вершины SP с направлением роста вниз;
  - схемы контроля пустого и полного состояния.
- блок общих операций и служебных подсистем:
  - централизованный параллельный арбитр общей шины данных;
  - контроллер прямого доступа к памяти;
  - блок предсказания переходов по схеме A4;
  - мультиплексоры адресов и данных для выбора активного источника обмена.

Арбитр шин, предсказатель переходов и контроллер прямого доступа к памяти рассматриваются как отдельные функциональные блоки. Они не заменяют устройство управления, а дополняют его: арбитр разрешает конфликты доступа к ОЗУ, предсказатель выдаёт предполагаемый адрес

следующей команды, а КППД выполняет обмен с памятью без участия арифметико-логического тракта процессора.

**Микро-ЭВМ помимо регистров общего назначения располагает специальными регистрами:**

- IP (instruction pointer) - 16-разрядный указатель команды, задающий адрес первого слова текущей или следующей команды в ПЗУ;
- IR0, IR1 (instruction register) - регистры двух слов команды; IR0 хранит код операции и номер регистра, IR1 хранит адресную часть;
- FR (flag register) - регистр флагов Z, S, C и O, отражающий состояние результата последней арифметико-логической операции;
- SP (stack pointer) - указатель вершины стека, изменяемый при выполнении PUSH и POP;
- GHR (global history register) - регистр глобальной истории переходов, используемый предсказателем;
- служебные регистры КППД - текущий адрес передачи, счётчик оставшихся байт и буфер принимаемого слова;
- служебные регистры кэша - теги, признаки действительности строк и счётчики давности хранения.

Специальные регистры не адресуются командами как обычные регистры R0-R11. Их содержимое задаётся аппаратной логикой соответствующих блоков и используется для хранения промежуточного состояния вычислительного процесса. Для хранения пользовательских данных должны использоваться РОН или ОЗУ данных, так как достоверность содержимого специальных регистров определяется текущей фазой выполнения команды.

## **1.2 Разработка системы команд**

**Количество реализованных команд - 11:**

- HLT - останов выполнения программы;
- MOV adr, Rn - чтение слова из памяти данных в регистр общего назначения;
- MOV Rn, adr - запись слова из регистра общего назначения в память данных;
- OR Rn, adr - побитовое ИЛИ регистра и слова памяти;
- NOR Rn, adr - инверсия результата побитового ИЛИ;
- SRA Rn - арифметический сдвиг регистра вправо;
- INCS Rn - инкремент регистра на значение флага S;

- PUSH Rn - занесение регистра в стек;
- POP Rn - извлечение слова из стека в регистр;
- JMP adr - безусловная загрузка адреса в IP;
- JZ adr - условный переход при установленном флаге Z.

Для кодирования одиннадцати команд достаточно четырёх бит, однако в проекте поле кода операции принято восьмибитным. Такое решение согласуется с 16-разрядной организацией слова команды, упрощает декодирование в VHDL и оставляет резерв кодов для возможного расширения системы команд. Неиспользуемые коды рассматриваются как зарезервированные и в аппаратной реализации переводят процессор в безопасное состояние останова.

Команда имеет фиксированную длину два 16-разрядных слова. Первое слово содержит код операции и номер регистра, второе слово используется как адрес операнда в ОЗУ, адрес перехода в ПЗУ или резервное поле для команд, которым адрес не требуется. Фиксированный формат выбран для того, чтобы фазовый автомат всегда выполнял одинаковую процедуру выборки IR0 и IR1.

#### **Положение полей команды фиксировано:**

- OPCODE[7..0] - код операции, всегда размещается в разрядах IR0(15..8);
- REG\_NUM[3..0] - номер регистра общего назначения, размещается в разрядах IR0(7..4);
- X[3..0] - служебное поле IR0(3..0), не влияющее на выполнение текущего набора команд;
- ADDR[15..0] - адресная часть IR1(15..0), используемая как адрес ОЗУ, адрес перехода в ПЗУ или резервное поле;
- RESERVED - коды и поля, оставленные для дальнейшего расширения системы команд.

Поле REG\_NUM имеет разрядность 4 бита, поэтому теоретически позволяет адресовать шестнадцать регистров. В проектируемой микро-ЭВМ фактически используются регистры R0-R11, а коды 12-15 не применяются в тестовой программе и считаются зарезервированными.

Адресное поле второго слова интерпретируется в зависимости от команды. Для MOV, OR и NOR оно задаёт адрес в ОЗУ данных; для JMP и JZ - адрес команды в ПЗУ; для HLT, SRA, INCS, PUSH и POP второе слово выбирается из ПЗУ вместе с командой, но при исполнении не используется.

#### **Кодировка КОП принята следующей:**

- 00h - HLT;
- 01h - MOV adr, Rn;
- 02h - MOV Rn, adr;
- 03h - OR Rn, adr;
- 04h - NOR Rn, adr;
- 05h - SRA Rn;
- 06h - INCS Rn;
- 07h - PUSH Rn;
- 08h - POP Rn;
- 09h - JMP adr;
- 0Ah - JZ adr.

Любой другой код операции считается зарезервированным. В аппаратной модели такой код не передаётся на выполнение произвольному блоку, а приводит процессор к безопасному останову, что упрощает отладку и защищает модель от случайного выполнения неопределённой команды.

Принятая система команд делится на четыре группы: команды обмена с памятью, арифметико-логические команды, стековые команды и команды передачи управления. Команда JZ включена для содержательной проверки предсказателя переходов: без условного перехода предсказатель невозможно показать в работе, так как безусловный JMP не требует анализа результата вычисления.

### **1.3 Описание взаимодействия всех блоков микро-ЭВМ при выполнении команд программы**

Главным блоком в иерархии микро-ЭВМ является устройство управления. Именно оно выполняет выборку двухсловной команды из ПЗУ, фиксирует слова команды в регистрах IR0 и IR1, декодирует код операции и номер регистра, а затем формирует последовательность управляющих сигналов для остальных узлов. Исполнительные блоки не работают независимо от устройства управления: они получают разрешение на чтение, запись или изменение состояния только в те такты, которые заданы фазовым автоматом T0...T5 и дополнительным тактом ожидания Tw.

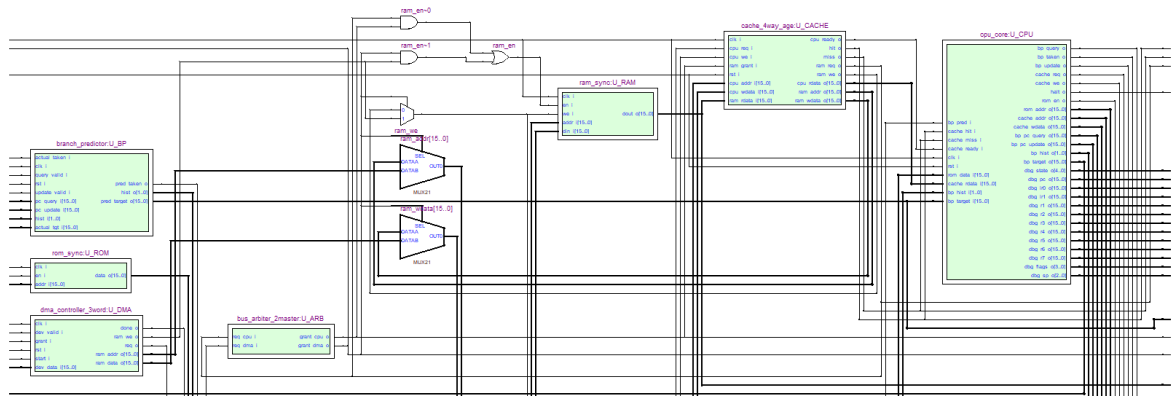
Непосредственно извне для работы микро-ЭВМ задаются тактовый сигнал CLK, сигнал сброса RESET, содержимое ПЗУ команд и начальные данные ОЗУ. Дальнейшее поведение системы определяется последовательностью команд программы. После декодирования команды устройство управления выбирает маршрут данных: обращение к памяти

через кэш, выполнение операции в АЛУ, обмен со стеком, изменение указателя команд или останов.

**К исполнительным и служебным блокам, взаимодействующим при выполнении программы, относятся:**

- блок общих операций: HLT, JMP, JZ, MOV adr, Rn и MOV Rn, adr;
- арифметико-логический блок: OR, NOR, SRA и INCS;
- блок стека: PUSH, POP, память стека и указатель SP;
- блок регистров общего назначения R0-R11 и регистр флагов FR;
- подсистема памяти: кэш данных, ОЗУ, ПЗУ команд и мультиплексоры адресов/данных;
- служебные подсистемы: арбитр общей шины, КПДП и предсказатель переходов A4.

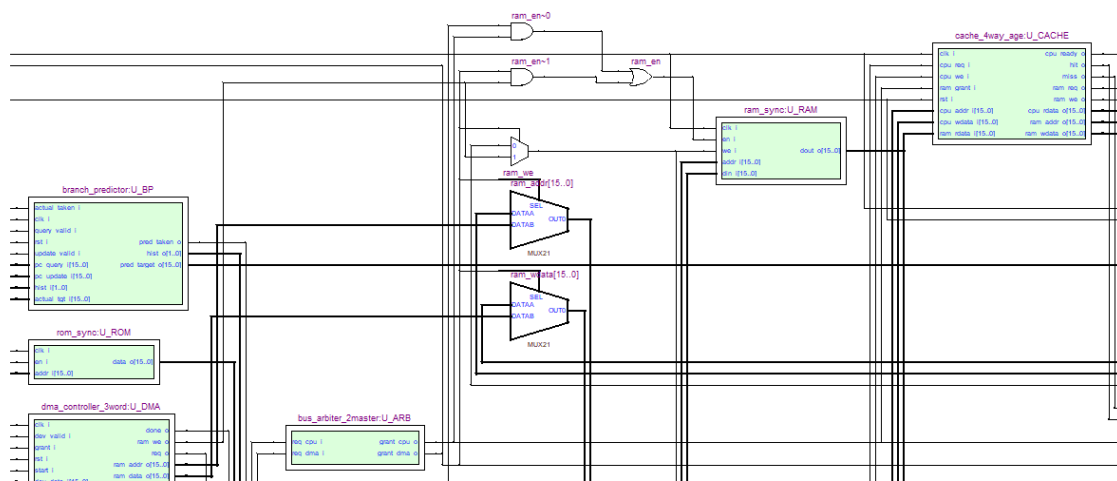
Общий RTL-фрагмент схемы, полученный после анализа и синтеза проекта в Quartus II, приведён на рисунке 1.1. На нём видны реальные экземпляры модулей верхнего уровня: процессорное ядро, кэш, ОЗУ, ПЗУ, КПДП, арбитр шины и предсказатель переходов. По этому фрагменту удобно проследить, что обмен данными внутри системы идёт не напрямую между всеми блоками, а через ограниченный набор общих линий и управляющих разрешений.



**Рисунок 1.1 - RTL-фрагмент взаимодействия основных блоков микро-ЭВМ в Quartus II**

При выполнении команд обмена с памятью процессорное ядро формирует адрес, направление обмена и слово данных. Запрос поступает в кэш данных. При попадании кэш возвращает требуемое слово без обращения к ОЗУ, при промахе или записи формируется обращение к синхронной памяти данных. Так как к ОЗУ может обращаться не только процессор, но и КПДП, доступ к общей памяти согласуется арбитром шины. В данной реализации арбитр выдаёт разрешающие сигналы `grant_cpu` и `grant_dma` и тем

самым не допускает одновременной записи разными источниками в одну и ту же память.



**Рисунок 1.2 - RTL-фрагмент взаимодействия кэша, ОЗУ, КПП и арбитра шины**

Команды PUSH и POP работают через стек глубиной 7 слов. При PUSH устройство управления разрешает чтение регистра и запись слова в текущую вершину стека, после чего изменяется указатель SP. При POP выполняется обратная операция: слово считывается из стека, записывается в выбранный регистр общего назначения, затем указатель вершины корректируется. Состояния пустого и полного стека контролируются аппаратно, чтобы исключить некорректное изменение памяти стека.

Команды передачи управления меняют содержимое IP. Для безусловного перехода JMP в IP загружается адрес из второго слова команды. Для условного перехода JZ дополнительно проверяется флаг Z. Предсказатель переходов A4 получает текущий адрес команды и историю переходов, выдаёт предполагаемый адрес следующей команды и обновляет свои таблицы после фактического выполнения перехода. Если предсказание оказалось неверным, устройство управления сбрасывает ошибочно выбранное направление и продолжает выборку с правильного адреса.

Таким образом, взаимодействие всех блоков организовано централизованно: устройство управления задаёт текущую фазу команды и активные разрешающие сигналы, исполнительные блоки выполняют только назначенную им часть операции, а арбитр и кэш ограничивают доступ к общей памяти. Такая структура уменьшает вероятность конфликтов на шинах и делает выполнение команд предсказуемым при моделировании и синтезе в Quartus II.

## 2 РАЗРАБОТКА ОСНОВНЫХ УСТРОЙСТВ МИКРО-ЭВМ

Во втором разделе общая структура микро-ЭВМ, описанная ранее, конкретизируется до уровня отдельных функциональных узлов. Основное внимание уделяется памяти, устройству управления и 16-разрядному арифметико-логическому устройству с блоком регистров общего назначения, поскольку именно эти элементы задают временную организацию процессора и определяют состав управляющих сигналов. Зафиксированные ниже решения не противоречат ТЗ и образуют основу, от которой уже можно переходить к построению схем и тестовых диаграмм.

### 2.1 Запоминающие устройства. Функциональный состав и временные диаграммы работы ОЗУ

Память команд реализуется как синхронное ПЗУ словной организации. При разрядности шины адреса 16 бит она может адресовать до 65536 слов по 16 бит. На входы ПЗУ подаются тактовый сигнал CLK, сигнал разрешения обращения CE\_ROM и адрес A\_ROM[15:0], формируемый указателем команд IP. Так как программа не изменяется в ходе работы, цикл записи для ПЗУ не предусматривается. По активному фронту такта при CE\_ROM = 1 адрес фиксируется, а 16-разрядное слово программы появляется на выходе DO\_ROM к следующему тактовому интервалу.

Поскольку в проекте принята фиксированная длина команды в два 16-разрядных слова, блок выборки последовательно обращается к ПЗУ по адресам IP и IP + 1. Первое считанное слово загружается в IR0 и содержит код операции и номер регистра, второе - в IR1 и содержит прямой адрес либо адрес перехода. Такая организация естественно согласуется с Гарвардской архитектурой, так как выборка команд не использует тракт памяти данных и не конфликтует с обменами по ОЗУ.

ОЗУ данных также принимается синхронным и словным, с тем же адресным пространством 65536 x 16. На его входы поступают CLK, CE\_RAM, RD\_RAM, WR\_RAM, адрес A\_RAM[15:0] и шина данных на запись DI\_RAM[15:0], на выходе формируется шина считанного слова DO\_RAM[15:0]. Одновременное разрешение чтения и записи не допускается. Физически ОЗУ обслуживает процессорную часть, кэш и контроллер прямого доступа к памяти, поэтому перед обращением к нему процессор должен получить разрешение от арбитра шины данных.

При чтении адрес и сигнал RD\_RAM выставляются в одном такте, а само слово становится доступным на DO\_RAM в следующем такте и далее может быть записано в РОН либо в буфер кэша. При записи адрес, данные DI\_RAM

и сигнал WR\_RAM должны быть стабильны к активному фронту, на котором слово заносится в память. Такой режим особенно удобен для сквозной записи кэша: при выполнении команды MOV reg, adr новое значение одним и тем же тактом попадает и в кэш, и в ОЗУ, что сохраняет согласованность копий.

**Таблица 6 - Сигналы ПЗУ и ОЗУ**

| Блок       | Сигнал       | Назначение                                           |
|------------|--------------|------------------------------------------------------|
| ПЗУ команд | CLK          | Тактовый вход синхронизации выборки команд.          |
| ПЗУ команд | CE_ROM       | Разрешение чтения слова программы.                   |
| ПЗУ команд | A_ROM[15:0]  | Адрес первого или второго слова текущей команды.     |
| ПЗУ команд | DO_ROM[15:0] | Выходное слово программы для загрузки в IR0 или IR1. |
| ОЗУ данных | CLK          | Тактовый вход синхронного чтения и записи.           |
| ОЗУ данных | CE_RAM       | Общее разрешение обращения к памяти данных.          |
| ОЗУ данных | RD_RAM       | Признак цикла чтения.                                |
| ОЗУ данных | WR_RAM       | Признак цикла записи.                                |
| ОЗУ данных | A_RAM[15:0]  | Адрес слова данных, выбранного CPU, кэшем или КПДП.  |
| ОЗУ данных | DI_RAM[15:0] | Шина данных на запись.                               |
| ОЗУ данных | DO_RAM[15:0] | Шина данных, считанных из памяти.                    |

Для временных диаграмм чтения из ОЗУ достаточно показать следующую последовательность: в такте  $T_n$  после выдачи гранта арбитром на шину подаются A\_RAM, CE\_RAM = 1 и RD\_RAM = 1; по следующему фронту память фиксирует адрес; в интервале  $T_{n+1}$  слово появляется на DO\_RAM и

может быть записано в буфер или регистр. Для диаграммы записи в такте  $T_n$  одновременно выставляются  $A\_RAM$ ,  $DI\_RAM$  и  $WR\_RAM = 1$ , а на активном фронте слово заносится в выбранную ячейку. Если при чтении произошел промах в кэше, фазовый автомат удерживает состояние ожидания до появления признака готовности памяти.

## 2.2 Устройство управления

Устройство управления целесообразно реализовать как синхронный конечный автомат с жесткой логикой. В регистре состояния автомата хранятся фазы цикла команды  $T_0$ - $T_5$  и дополнительное состояние ожидания  $T_w$ . На входы автомата поступают код операции из  $IR_0$ , флаги  $Z$ ,  $C$ ,  $S$  и  $O$  из регистра  $FR$ , признаки попадания в кэш, сигналы  $grant$  от арбитра, а также служебные признаки переполнения и опустошения стека. На выходах формируются управляющие воздействия на ПЗУ, ОЗУ, блок РОН, АЛУ, стек,  $IP$ ,  $IR$  и вспомогательные ускоряющие подсистемы.

Центральный узел распределения управляющих сигналов декодирует поле кода операции и определяет класс команды: обмен с памятью, операция АЛУ, работа со стеком, переход или останов. Локальный узел управления фазами команды на основании этого класса выбирает требуемую последовательность микроопераций. Для коротких регистровых операций  $INCS$ ,  $OR$ ,  $NOR$  и  $SRA$  используется стандартный маршрут без дополнительных ожиданий. Для команд обращения к данным возможна вставка тактов  $T_w$ , если шина занята КППДП или если кэш не содержит нужной строки.

Блок указателя команд  $IP$  представляет собой 16-разрядный регистр с инкрементатором на два адреса и мультиплексором следующего значения. На его вход подаются как минимум четыре задания: адрес первой команды после сброса, естественное продолжение  $IP + 2$ , адрес из регистра  $IR_1$  и адрес, предложенный системой предсказания переходов. Для линейных команд в конце цикла загружается  $IP + 2$ . Для  $JMP$  безусловно используется адрес из  $IR_1$ . Для  $JZ$  сначала может быть выбран предсказанный адрес, а после проверки условия в конце цикла производится окончательная коррекция.

Регистр кода команд удобно разделить на два слова  $IR_0$  и  $IR_1$ . В  $IR_0$  хранятся код операции и номера регистров, а в  $IR_1$  - адресная часть команды. Такое разделение уменьшает длину соединений на схеме и упрощает декодирование. Блок управления стековой памятью включает регистр  $SP$ , схему сравнения с границами стека и сигналы  $SP\_DEC$ ,  $SP\_INC$ ,  $STK\_WE$ ,

STK\_OE. Для PUSH сначала уменьшается SP, затем выполняется запись слова в стек. Для POP сначала считывается слово по адресу SP, затем после загрузки регистра SP увеличивается.

**Таблица 7 - Основные управляющие сигналы устройства управления**

| Сигнал              | Приемник   | Фазы  | Назначение                                          |
|---------------------|------------|-------|-----------------------------------------------------|
| ROM_CE              | ПЗУ        | T0-T1 | Разрешение выборки первого и второго слова команды. |
| IR0_LD              | IR0        | T1    | Захват первого слова команды.                       |
| IR1_LD              | IR1        | T2    | Захват второго слова команды.                       |
| IP_SEL[1:0]         | MUX IP     | T5    | Выбор следующего адреса команды.                    |
| IP_LD               | IP         | T5    | Загрузка нового значения указателя команд.          |
| RF_ASEL,<br>RF_BSEL | Блок РОН   | T3    | Выбор регистров-источников по полям команды.        |
| RF_WE               | Блок РОН   | T5    | Разрешение записи результата в регистр.             |
| ALU_OP[2:0]         | АЛУ        | T4    | Код операции INCS, OR, NOR или SRA.                 |
| FR_WE               | Регистр FR | T5    | Фиксация обновленных флагов.                        |
| CACHE_LOOK<br>UP    | Кэш        | T3-T4 | Запуск поиска строки по адресу обращения.           |
| CACHE_FILL          | Кэш        | Tw-T5 | Заполнение или                                      |

|                       |                 |        |                                                   |
|-----------------------|-----------------|--------|---------------------------------------------------|
|                       |                 |        | обновление строки после промаха.                  |
| RAM_RD                | ОЗУ             | T4, Tw | Инициация синхронного чтения из ОЗУ.              |
| RAM_WR                | ОЗУ             | T4     | Инициация синхронной записи в ОЗУ.                |
| SP_DEC,<br>SP_INC     | Регистр SP      | T4-T5  | Изменение указателя стека для PUSH и POP.         |
| STK_WE,<br>STK_OE     | Стековая память | T4-T5  | Запись или чтение слова стека.                    |
| PRED_REQ,<br>PRED_UPD | Предсказатель   | T2, T5 | Запрос прогноза и обновление его состояния.       |
| CPU_HALT              | Фазовый автомат | T5     | Перевод автомата управления в состояние останова. |

Для разработки временных диаграмм удобно зафиксировать базовую фазовую организацию цикла. В фазе T0 в ПЗУ выставляется адрес IP, в фазе T1 первое слово команды заносится в IR0 и одновременно запрашивается слово по адресу  $IP + 1$ , в фазе T2 второе слово записывается в IR1 и производится первичное декодирование. Фаза T3 используется для чтения операндов из РОН и запуска сервисных блоков, фаза T4 - для выполнения операции или обмена с памятью, фаза T5 - для записи результата, обновления флагов и окончательной загрузки IP. При промахе в кэше или занятой шине между T4 и T5 вставляется состояние Tw, которое удерживает уже выбранные адреса и управляющие коды до появления сигнала готовности.

### 2.3 Арифметико-логическое устройство и блок регистров общего назначения

Арифметико-логическое устройство выполняет операции INCS, OR, NOR и SRA над 16-разрядными словами. Операнды поступают из блока регистров общего назначения или из тракта данных памяти, а результат возвращается в выбранный регистр через внутреннюю шину процессора.

Входные сигналы АЛУ:  $A[15:0]$  и  $B[15:0]$  — операнды; SFLAG — текущее значение признака знака для операции INCS;  $ALU\_OP[2:0]$  — код выполняемой операции. Выходные сигналы:  $Y[15:0]$  — результат; Z, S, C, O — признаки нуля, знака, переноса/вытесненного бита и переполнения.

Для операции SRA сохраняется старший бит исходного операнда, а младший бит заносится во флаг C. Для OR и NOR используются оба операнда, при этом C и O сбрасываются. Для INCS к операнду прибавляется текущее значение флага S, поэтому при  $S = 0$  регистр не изменяется, а при  $S = 1$  увеличивается на единицу.

Электрическая схема арифметико-логического устройства приведена в графической части проекта.

Рис. 2.3 - Арифметико-логическое устройство.

## 2.4. Блок регистров общего назначения

Блок регистров общего назначения реализуется как регистровый файл из 12 регистров по 16 бит с двумя портами чтения и одним портом записи. Поля команды кодируются четырьмя битами, поэтому коды 12...15 остаются резервными. Двухпортовая организация чтения позволяет за один такт одновременно извлекать оба операнда для OR и NOR, тогда как команды INCS и SRA используют только один регистр. Для упрощения фазового автомата чтение РОН удобно считать комбинационным, а запись - синхронной по фронту такта при активном сигнале RF\_WE в конце выполнения команды.

АЛУ строится как 16-разрядный комбинационный блок, состоящий из входных мультиплексоров, сумматора, логического узла OR/NOR, арифметического сдвигателя вправо и мультиплексора результата Y. На вход A подается содержимое регистра Rd. На вход B в зависимости от кода  $ALU\_OP$  поступает либо содержимое регистра Rs, либо одноразрядное значение флага S, расширенное до 16 бит как 0000...0S. Для SRA используется только вход A, а вход B в этой операции несущественен. После формирования результата Y он записывается в регистр-приемник, а набор флагов передается в регистр FR.

Операция INCS требует отдельного пояснения. В ней ко второму входу сумматора подается не константа 1, а текущее значение знакового флага S. Если  $S = 0$ , регистр Rd не изменяется. Если  $S = 1$ , значение Rd увеличивается

на единицу. Для OR и NOR используются оба порта блока РОН. Операция NOR удобно реализуется как обычное OR с последующей инверсией результата. Для SRA знаковый бит сохраняется, а вытесненный младший разряд исходного операнда записывается во флаг C. Флаг Z во всех арифметических и логических операциях принимает единичное значение при нулевом результате, а флаг S копирует старший бит результата.

**Таблица 8 - Реализация операций АЛУ и обновление флагов**

| Операция   | Аппаратное выражение                   | Запись результата | Обновление флагов                                                              |
|------------|----------------------------------------|-------------------|--------------------------------------------------------------------------------|
| INCS Rn    | $Y = A + \{15 \times 0, \text{FR.S}\}$ | $Rn \leftarrow Y$ | $Z = (Y = 0)$ ; $S = Y15$ ; $C = \text{перенос}$ ; $O = \text{переполнение}$ . |
| OR Rs, Rd  | $Y = A \text{ OR } B$                  | $Rd \leftarrow Y$ | Обновляются Z и S; $C = 0$ ; $O = 0$ .                                         |
| NOR Rs, Rd | $Y = \text{NOT}(A \text{ OR } B)$      | $Rd \leftarrow Y$ | Обновляются Z и S; $C = 0$ ; $O = 0$ .                                         |
| SRA Rd     | $Y15 = A15$ ; $Yi = A(i+1)$            | $Rd \leftarrow Y$ | $Z = (Y = 0)$ ; $S = Y15$ ; $C = A0$ ; $O = 0$ .                               |

Таким образом, блок РОН и 16-разрядное АЛУ полностью покрывают все вычислительные операции технического задания и обеспечивают формирование обязательного регистра флагов. Для раздела 3.1 по этим узлам достаточно подготовить отдельные тесты, демонстрирующие как минимум INCS при  $S = 0$  и  $S = 1$ , побитовое OR, побитовое NOR и арифметический сдвиг вправо отрицательного числа. Эти же тесты одновременно подтвердят корректность изменения флагов Z, C, S и O.

## 2.4 Организация кэш-памяти процессора

Кэш-память располагается между исполнительной частью процессора и ОЗУ данных и предназначена для уменьшения среднего времени доступа к данным. По техническому заданию для разрабатываемой микро-ЭВМ требуется кэш с параметром  $k =$ , алгоритмом замещения по наибольшей давности хранения и синхронизацией с памятью по схеме «сквозная запись с отображением». Для дальнейшей разработки принимается четырёхканальный

множественно-ассоциативный кэш с 16 наборами и длиной строки в одно 16-разрядное слово. Такое решение упрощает схему сравнения тегов, не требует выделения поля смещения внутри строки и хорошо подходит для учебного моделирования.

При 16 наборах адрес данных  $A[15:0]$  делится на две части: младшие четыре бита  $A[3:0]$  образуют индекс набора, а старшие двенадцать бит  $A[15:4]$  записываются в поле тега. В каждом наборе имеются четыре строки, каждая из которых содержит слово данных, тег, признак действительности и поле возраста. Поскольку используется сквозная запись, хранить признак модификации *dirty* не требуется: любое изменение слова немедленно дублируется в ОЗУ.

### Структура строки кэша

**Таблица 9 - Состав строки кэша данных**

| Поле строки | Разрядность | Назначение                                                                                          |
|-------------|-------------|-----------------------------------------------------------------------------------------------------|
| V           | 1 бит       | Признак того, что строка содержит корректные данные и может участвовать в сравнении тегов.          |
| TAG         | 12 бит      | Старшие биты адреса $A[15:4]$ , используемые для определения попадания в выбранном наборе.          |
| DATA        | 16 бит      | Кэшируемое слово данных, выдаваемое процессору при попадании.                                       |
| AGE         | 2 бита      | Код относительной давности хранения в пределах набора; используется при выборе строки на замещение. |

Попадание в кэш определяется следующим образом. По индексу  $A[3:0]$  выбирается нужный набор, после чего тег адреса одновременно сравнивается с четырьмя тегами всех строк набора. Если хотя бы в одной строке  $V = 1$  и TAG совпадает с  $A[15:4]$ , формируется сигнал *hit*, а на выход кэша через мультиплексор поступает соответствующее поле DATA. При отсутствии совпадений формируется *miss* и контроллер кэша запрашивает доступ к ОЗУ через арбитр шины.

Алгоритм замещения по наибольшей давности хранения реализуется проще, чем LRU, поскольку возраст строки меняется только в момент загрузки новой строки, а не при каждом чтении. Если в наборе имеется хотя бы одна недействительная строка, новая запись помещается в неё. Если набор полностью заполнен, выбирается строка с максимальным значением AGE. После загрузки новой строки её возраст устанавливается равным нулю, а возраста остальных действительных строк набора увеличиваются на единицу с насыщением до максимума. При попадании значения AGE не меняются, так как критерий замещения связан не с последним обращением, а именно с длительностью хранения строки в кэше.

Политика «сквозная запись с отображением» трактуется следующим образом. При записи по адресу, который уже присутствует в кэше, слово одновременно обновляется и в строке кэша, и в ОЗУ. При записи по адресу, отсутствующему в кэше, слово сначала передаётся в ОЗУ, после чего соответствующая строка создаётся и в кэше. Тем самым после любой операции записи кэш не расходится с основной памятью, а последующие чтения недавно записанного слова могут выполняться с попаданием.

С точки зрения временных диаграмм чтение при попадании укладывается в один цикл обращения к кэшу: в текущем такте выставляется адрес, а к концу такта или в следующем синхронном такте на выходе формируется слово данных. При промахе добавляются такты арбитража и чтения ОЗУ, после чего загруженная строка сразу же выдаётся в процессор. Для записи время операции в основном определяется синхронным ОЗУ, потому что даже при попадании в кэш должно быть выполнено дублирование данных в оперативную память.

## 2.5 Описание системы предсказания переходов

Для технического задания задана схема предсказания переходов A4 с шаблоном «4, PC(2) || GHR(2)». В пояснительной записке целесообразно реализовать её как двухуровневый адаптивный предсказатель, использующий два младших бита адреса условного перехода и двухразрядный регистр глобальной истории. Такая структура хорошо согласуется с параметрами задания и при этом достаточно проста для моделирования на ПЛИС.

В состав блока предсказания включаются: регистр глобальной истории GHR[1:0], таблица историй шаблонов PHT на 16 ячеек, а также буфер целевых адресов BTV на 16 записей. Индекс таблицы образуется конкатенацией двух младших бит адреса команды перехода и текущего содержимого GHR: INDEX

= PC[1:0] || GHR[1:0]. По этому индексу выбирается 2-битный счётчик насыщения, задающий прогноз направления перехода.

### Состояния счётчика предсказания

**Таблица 10 - Интерпретация 2-битного счётчика предсказания**

| Код | Состояние         | Выдаваемый прогноз |
|-----|-------------------|--------------------|
| 00  | сильно не переход | не переходить      |
| 01  | слабо не переход  | не переходить      |
| 10  | слабо переход     | переходить         |
| 11  | сильно переход    | переходить         |

В проектируемой системе условным переходом считается команда JZ adr, введённая как дополнение к обязательному набору команд. Для неё предсказатель решает две задачи: заранее выбрать предполагаемое направление и, в случае прогноза «переход», выдать адрес цели. Поскольку адрес перехода хранится во втором слове команды, для ранней подстановки нового значения IP в блоке предсказания дополнительно используется ВТВ. В каждой записи ВТВ хранятся признак действительности, тег адреса ветвления и ранее подтверждённый целевой адрес.

Во время выборки команды блок предсказания получает текущее значение IP. Если ВТВ содержит запись для данного адреса ветвления, а соответствующий счётчик РНТ находится в состоянии 10 или 11, формируется прогноз «переход» и в IP подставляется целевой адрес из ВТВ. В противном случае выбирается последовательное продолжение программы, то есть IP + 2, так как длина команды принята равной двум словам. Для безусловной команды JMP adr необходимость в предсказании направления отсутствует: она всегда считается выполняющейся с переходом, и её целевой адрес просто загружается в IP после декодирования.

После фактического выполнения условного перехода устройство управления возвращает в блок предсказания сигнал ACTUAL\_TAKEN. По нему выбранный счётчик в РНТ увеличивается или уменьшается на единицу до ближайшего предельного состояния, а регистр глобальной истории обновляется по правилу  $GHR\_next = \{GHR[0], ACTUAL\_TAKEN\}$ . Если предсказание оказалось неверным, устройство управления сбрасывает ошибочно выбранную следующую команду, корректирует значение IP и

обновляет ВТВ новым адресом цели. В результате на часто повторяющихся ветвлениях предсказатель постепенно подстраивается к характерному шаблону программы и уменьшает число пустых циклов на переходах.

## 2.6 Аппаратура и функционирование КПДП

Контроллер прямого доступа к памяти предназначен для обмена данными между внешним устройством и ОЗУ без участия арифметико-логического тракта процессора. Для технического задания заданы начальный адрес 10 и объём блока 6 байт. Поскольку в проекте принята словная организация ОЗУ по 16 бит, это соответствует передаче трёх последовательных слов по адресам 10, 11 и 12. Такое фиксированное задание удобно для моделирования, потому что позволяет заранее задать регистры адреса и счётчика и затем наблюдать корректность заполнения памяти по дампу.

В состав КПДП входят регистр начального адреса `START_ADDR`, рабочий регистр текущего адреса `CURR_ADDR`, счётчик оставшегося объёма `BYTE_CNT`, входной регистр данных `DATA_BUF`, регистр состояния `STATUS` и конечный автомат управления. Перед началом передачи в `START_ADDR` загружается значение 10, а в `BYTE_CNT` - значение 6. После поступления внешнего запроса контроллер копирует начальный адрес в `CURR_ADDR`, переводит `STATUS` в состояние `BUSY` и запрашивает у арбитра доступ к системной шине данных.

### Основные регистры и сигналы КПДП

**Таблица 11 - Регистры и сигналы контроллера прямого доступа к памяти**

| Элемент                 | Разрядность/<br>начальное<br>значение | Назначение                                                                                        |
|-------------------------|---------------------------------------|---------------------------------------------------------------------------------------------------|
| <code>START_ADDR</code> | 16 бит / 10                           | Хранит базовый адрес области ОЗУ, в которую должен быть записан принимаемый блок.                 |
| <code>CURR_ADDR</code>  | 16 бит                                | Содержит текущий адрес очередной передачи; увеличивается после каждого успешно записанного слова. |
| <code>BYTE_CNT</code>   | 16 бит / 6                            | Содержит число ещё не переданных байтов; после каждой 16-битной                                   |

|                                     |                      |                                                                               |
|-------------------------------------|----------------------|-------------------------------------------------------------------------------|
|                                     |                      | передачи уменьшается на 2.                                                    |
| DATA_BUF                            | 16 бит               | Буферизует слово, принимаемое от внешнего устройства до момента записи в ОЗУ. |
| STATUS                              | несколько бит        | Формирует признаки BUSY, DONE и, при необходимости, ERROR.                    |
| DREQ,<br>DACK,<br>BR_DMA,<br>BG_DMA | служебные<br>сигналы | Организуют внешний запрос на передачу и захват системной шины через арбитр.   |

В данной работе КПДП рассматривается в режиме передачи от внешнего источника в ОЗУ. Алгоритм его работы таков. Сначала внешнее устройство формирует сигнал DREQ и выдаёт первое 16-разрядное слово на вход DATA\_BUF. Контроллер выставляет запрос BR\_DMA на арбитр шины. После получения разрешения BG\_DMA он подаёт адрес CURR\_ADDR на вход ОЗУ, выдаёт данные из DATA\_BUF на шину DI\_RAM и формирует сигнал записи WR\_RAM. После завершения такта записи CURR\_ADDR увеличивается на единицу, а BYTE\_CNT уменьшается на 2. Цикл повторяется до тех пор, пока счётчик не достигнет нуля.

Так как объём блока составляет всего 6 байт, для упрощения управляющей логики выбирается пакетный режим: после получения гранта контроллер удерживает шину до завершения всех трёх слов передачи. По окончании обмена признак BUSY сбрасывается, формируется DONE, а арбитру передаётся освобождение шины. Если требуется прерывание процессора по завершении обмена, сигнал DONE может быть дополнительно подан на вход контроллера прерываний или на отдельный флаг состояния.

Использование КПДП особенно удобно при моделировании, поскольку позволяет показать, что память данных изменяется даже без выполнения процессором явных команд MOV reg, adr. В дампе памяти после окончания передачи должны измениться ячейки с адресами 10, 11 и 12, а содержимое регистров процессора при этом может остаться неизменным. Именно эта особенность и демонстрирует принцип прямого доступа к памяти как самостоятельного шино-мастеринга со стороны внешнего контроллера.

## 2.7 Описание арбитра шины

Поскольку архитектура микро-ЭВМ является Гарвардской, выборка команд из ПЗУ выполняется по выделенному тракту и не требует участия общего арбитра данных. Следовательно, арбитраж необходим только для обращений к ОЗУ данных. Основными претендентами на использование этой шины в проекте являются процессорная подсистема данных, представленная контроллером кэша, и контроллер прямого доступа к памяти. По условию задания должен быть реализован централизованный параллельный арбитраж, то есть все запросы одновременно поступают в один управляющий узел, который формирует взаимоисключающие сигналы разрешения.

Для данной микро-ЭВМ достаточно двух входов запроса: REQ\_CPU и REQ\_DMA. Сигнал REQ\_CPU формируется контроллером кэша при промахе чтения или при сквозной записи в ОЗУ. Сигнал REQ\_DMA выдаётся КППД на время пакетной передачи. Арбитр реализуется как комбинационный приоритетный блок с регистрацией результата в такте синхронизации. В принятом проектном решении более высокий приоритет получает REQ\_DMA, поскольку источник данных может быть внешним устройством реального времени, а объём обмена всего 6 байт, поэтому задержка центрального процессора будет небольшой.

### Правила выдачи грантов

**Таблица 12 - Логика централизованного параллельного арбитра для шины данных**

| REQ_DMA | REQ_CPU | GNT_DMA | GNT_CPU |
|---------|---------|---------|---------|
| 0       | 0       | 0       | 0       |
| 0       | 1       | 0       | 1       |
| 1       | 0       | 1       | 0       |
| 1       | 1       | 1       | 0       |

Из таблицы видно, что при одновременном поступлении двух запросов приоритет получает КППД. Это означает, что процессорная подсистема должна удерживать свой запрос до освобождения шины. После завершения пакетной передачи DMA арбитр снимает GNT\_DMA и в следующем такте может выдать разрешение процессору. Благодаря выделенному тракту команд даже во время активного КППД содержимое указателя команд IP и выборка

команд из ПЗУ не обязательно останавливаются, однако фактические обращения к ОЗУ данных со стороны процессора задерживаются до освобождения шины.

Централизованный параллельный арбитр удобен тем, что его схема легко описывается логическими формулами. Для выбранной двухмастерной конфигурации они имеют вид:  $GNT\_DMA = REQ\_DMA$ ,  $GNT\_CPU = REQ\_CPU \text{ and not } REQ\_DMA$ . При необходимости расширения системы к арбитру можно добавить новые входы запросов и перераспределить приоритеты, не меняя принципа работы уже существующих блоков. Это делает принятое решение удобной основой для последующего развития архитектуры.

Таким образом, во втором разделе получена уже не только общая, но и структурно детализированная архитектура микро-ЭВМ. Для памяти, АЛУ, устройства управления, кэша, предсказателя переходов, КПП и арбитра шины сформулированы конкретные проектные решения, которые далее могут быть непосредственно перенесены в схемы Quartus и использованы при подготовке временных диаграмм и тестовых моделей.

### **3. Функциональное моделирование**

В данном разделе будут представлены временные диаграммы разработанных блоков, а так же всей смоделированной системы сразу

### 3 ФУНКЦИОНАЛЬНОЕ МОДЕЛИРОВАНИЕ

Функциональное моделирование предназначено для подтверждения корректности логики отдельных блоков и их совместной работы в составе полной микро-ЭВМ. На данном этапе удобно использовать поведенческие модели без учёта реальных задержек распространения сигналов, оставляя только общий тактовый сигнал CLK и синхронный сброс RESET. Перед запуском тестов принимаются одинаковые начальные условия: регистры общего назначения R0-R11 обнулены, регистр флагов FR содержит 0000, указатель команд IP установлен в 0000h, указатель стека SP равен 7, все признаки валидности строк кэша сброшены, регистр глобальной истории GHR установлен в 00, а все счётчики РНТ инициализированы состоянием 01 («слабо не переходить»).

Такой способ подготовки модели важен по двум причинам. Во-первых, он делает результаты повторяемыми: любой запуск тестбенча приводит к одним и тем же переходным процессам. Во-вторых, он позволяет заранее вычислить контрольные значения регистров и памяти и затем сравнить их с тем, что будет видно на временных диаграммах Quartus или ModelSim. Ниже приведены контрольные сценарии как для автономной проверки блоков, так и для сквозного моделирования всей микро-ЭВМ по тестовой программе.

#### 3.1 Моделирование отдельных блоков

Автономное моделирование блоков позволяет убедиться, что каждый функциональный узел работает правильно ещё до сборки полной схемы. Для каждого testbench рекомендуется подавать минимально необходимый набор входных воздействий и фиксировать не только конечный результат, но и порядок изменения внутренних сигналов. Это особенно важно для синхронных блоков, где ошибка часто проявляется не в самом значении, а в неверном такте его появления.

**Таблица 13 - Контрольные тесты для моделирования отдельных блоков**

| Блок                    | Тестовое воздействие                                                                                    | Контролируемые сигналы                  | Ожидаемый результат                                                                                 |
|-------------------------|---------------------------------------------------------------------------------------------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------|
| ПЗУ и регистр команд IR | Последовательно подать адреса 0000h, 0001h, 0002h и 0003h при CE_ROM = 1; по фронтам такта активировать | A_ROM[15:0], DO_ROM[15:0], IR0, IR1, IP | В IR0/IR1 должны последовательно загрузиться пары 0110h/0020h и 0510h/0000h; после завершения цикла |

|                |                                                                                                                                            |                                                         |                                                                                                                                                                                                      |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                | IR0_LD и IR1_LD.                                                                                                                           |                                                         | IP увеличивается на 2.                                                                                                                                                                               |
| ОЗУ данных     | Сначала выполнить чтение по адресу 0021h, затем запись C001h по адресу 0030h и повторное чтение этой ячейки.                               | A_RAM[15:0], RD_RAM, WR_RAM, DI_RAM[15:0], DO_RAM[15:0] | При первом чтении на выходе DO_RAM появляется 00F0h, после цикла записи и повторного чтения - значение C001h.                                                                                        |
| Блок РОН и АЛУ | Задать R1 = 8001h, FR.S = 1, R2 = 00F0h и значения памяти M[0022] = 0F0Fh, M[0023] = 0000h; последовательно выполнить SRA, INCS, OR и NOR. | RF_WE, ALU_OP, A[15:0], B[15:0], Y[15:0], Z, S, C, O    | SRA формирует C000h и C = 1; INCS даёт C001h; OR формирует 0FFFh; NOR формирует F000h. Для отдельной проверки INCS полезно также задать FR.S = 0 и убедиться, что содержимое регистра не изменяется. |
| Стек           | При начальном SP = 7 выполнить PUSH для слова C001h, а затем POP в приёмный регистр.                                                       | SP, STK_WE, STK_OE, STK_DIN[15:0], STK_DOUT[15:0]       | После PUSH указатель уменьшается до 6 и в ячейке STK[6] записывается C001h; после POP значение попадает в регистр, а SP возвращается к 7.                                                            |
| Кэш-память     | Дважды обратиться к адресу 0025h, затем выполнить запись по адресу 0030h. Для проверки вытеснения                                          | ADDR, HIT, MISS, TAG_HIT, LINE_WE, AGE[1:0], RAM_REQ    | Первое чтение 0025h должно дать miss, второе - hit. При записи в 0030h обновляются и                                                                                                                 |

|                         |                                                                                                                                            |                                                                       |                                                                                                                                                                                                                                                   |
|-------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                         | дополнительно последовательно читать 0010h, 0110h, 0210h, 0310h и 0410h.                                                                   |                                                                       | кэш, и ОЗУ. Пятое обращение к одному и тому же набору вызывает вытеснение наиболее старой строки.                                                                                                                                                 |
| Предсказатель переходов | В тестбенче вручную задавать пары PC[1:0] и GHR[1:0], а также текущие состояния счётчиков PHT; затем подавать фактический исход ветвления. | PRED_TAKEN, PHT_STATE[1:0], GHR[1:0], BTB_HIT, BR_RESOLVE             | Прогноз определяется старшим битом двухразрядного счётчика. После разрешения перехода счётчик изменяется по насыщаемому закону $00 \leftrightarrow 01 \leftrightarrow 10 \leftrightarrow 11$ , а GHR сдвигается с занесением фактического исхода. |
| КПДП и арбитр           | Одновременно подать REQ_CPU = 1 и REQ_DMA = 1, затем инициировать DREQ и поочередно загрузить в DATA_BUF слова 1111h, 2222h и 3333h.       | REQ_CPU, REQ_DMA, GNT_CPU, GNT_DMA, CURR_ADDR, BYTE_CNT, WR_RAM, DONE | Арбитр должен выдать приоритет DMA. После трёх циклов записи в ОЗУ по адресам 000Ah-000Ch формируется DONE, шина освобождается и доступ может получить процессор.                                                                                 |
| Устройство управления   | Подавать в IR0 коды команд 01h, 05h, 07h, 09h и 0Ah при разных флагах; отслеживать последовательность                                      | STATE, ROM_CE, IR0_LD, IR1_LD, RAM_RD, RAM_WR, RF_WE,                 | Для каждой команды должна формироваться своя корректная последовательность управляющих                                                                                                                                                            |

|  |                                    |                   |                                                                                                             |
|--|------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------|
|  | фаз T0-T5 и состояние ожидания Tw. | STK_WE,<br>IP_SEL | сигналов;<br>состояние Tw<br>появляется только<br>при промахе в<br>кэше либо при<br>занятой шине<br>данных. |
|--|------------------------------------|-------------------|-------------------------------------------------------------------------------------------------------------|

Из приведённых сценариев наиболее показательны тесты АЛУ, кэша и предсказателя переходов. Для АЛУ важно вывести на диаграмму сразу и операнды, и флаги, поскольку неверное обновление Z, S или C приводит к каскадным ошибкам в команде JZ и в операции INCS. Для кэша, помимо факта hit/miss, необходимо показать изменение полей возраста, иначе не удастся доказать правильность алгоритма замещения по наибольшей давности хранения.

Проверка предсказателя переходов в составе отдельного блока полезна тем, что позволяет не смешивать его динамику с остальными сигналами процессора. В автономном testbench проще проследить, как комбинация PC(2)||GHR(2) выбирает строку PHT, как формируется прогноз и как после разрешения ветвления обновляются и счётчик, и регистр GHR. Аналогично сценарий для КПДП и арбитра должен обязательно содержать одновременный запрос процессора и DMA, потому что только в этом случае проявляется требование задания о централизованном параллельном арбитраже с приоритетом контроллера прямого доступа к памяти.

## 3.2 Моделирование всей системы

После проверки отдельных узлов выполняется сквозное моделирование полной микро-ЭВМ. Для него в ПЗУ загружается тестовая программа, охватывающая все команды системы, а ОЗУ и сервисные блоки инициализируются контрольными значениями. Дополнительно в процессе моделирования создаётся внешний запрос DREQ, чтобы продемонстрировать работу КПДП и арбитра уже в составе общей структуры, а не только в изолированном тестбенче.

### 3.2.1 Тестовая программа и исходные данные

Тестовая программа составлена так, чтобы в одном проходе задействовать загрузку из памяти, запись в память, логические операции, арифметический сдвиг, стековые команды, условный и безусловный переходы, а также останов. Адреса в таблице 14 относятся к пространству ПЗУ и являются адресами слов программы. Для всех регистровых команд второе

слово команды сохраняется как 0000h, поскольку в проекте принят фиксированный двухсловный формат.

**Таблица 14 - Листинг тестовой программы для полной модели**

| <b>Адрес ПЗУ</b> | <b>Слово 0</b> | <b>Слово 1</b> | <b>Мнемоника</b> | <b>Назначение</b>                                               |
|------------------|----------------|----------------|------------------|-----------------------------------------------------------------|
| 0000h            | 0110h          | 0020h          | MOV 0020h, R1    | Загрузка отрицательного числа для проверки SRA.                 |
| 0002h            | 0510h          | 0000h          | SRA R1           | Арифметический сдвиг вправо; ожидается результат C000h и C = 1. |
| 0004h            | 0610h          | 0000h          | INCS R1          | При S = 1 значение регистра увеличивается до C001h.             |
| 0006h            | 0710h          | 0000h          | PUSH R1          | Помещение слова C001h в стек.                                   |
| 0008h            | 0120h          | 0021h          | MOV 0021h, R2    | Загрузка первого операнда для OR и NOR.                         |
| 000Ah            | 0320h          | 0022h          | OR R2, 0022h     | Формирование значения 0FFFh.                                    |
| 000Ch            | 0420h          | 0023h          | NOR R2, 0023h    | Формирование значения F000h.                                    |
| 000Eh            | 0830h          | 0000h          | POP R3           | Извлечение C001h из стека в регистр R3.                         |
| 0010h            | 0230h          | 0030h          | MOV R3, 0030h    | Запись результата в память данных.                              |
| 0012h            | 0140h          | 0025h          | MOV 0025h, R4    | Загрузка FFFFh для последующего формирования нуля.              |
| 0014h            | 0440h          | 0025h          | NOR R4, 0025h    | Получение 0000h и установка флага Z = 1.                        |

|       |       |       |               |                                                     |
|-------|-------|-------|---------------|-----------------------------------------------------|
| 0016h | 0A00h | 001Ah | JZ 001Ah      | Условный переход на адрес 001Ah при Z = 1.          |
| 0018h | 0150h | 0024h | MOV 0024h, R5 | Эта команда должна быть пропущена.                  |
| 001Ah | 0900h | 001Eh | JMP 001Eh     | Безусловный переход к чтению результата DMA.        |
| 001Ch | 0160h | 0024h | MOV 0024h, R6 | Эта команда также должна быть пропущена.            |
| 001Eh | 0170h | 000Ah | MOV 000Ah, R7 | Чтение первого слова, записанного контроллером DMA. |
| 0020h | 0000h | 0000h | HLT           | Останов полной модели.                              |

Принятый листинг удобно использовать и как программу, и как дамп ПЗУ. При необходимости двоичное представление слов получается прямым переводом соответствующих hex-кодов в 16-разрядную форму; например, слово 0110h соответствует коду 0000 0001 0001 0000b. Для инициализации ОЗУ данных используются значения, приведённые в таблице 15. Важно не путать одинаковые числовые адреса в таблицах 14 и 15: в первом случае они относятся к пространству команд, во втором - к пространству данных, что соответствует Гарвардской архитектуре.

**Таблица 15 - Контрольный дамп ОЗУ до и после выполнения программы**

| Адрес данных | До запуска | После HLT | Пояснение                                     |
|--------------|------------|-----------|-----------------------------------------------|
| 000Ah        | 0000h      | 1111h     | Первое слово, переданное КПДП.                |
| 000Bh        | 0000h      | 2222h     | Второе слово, переданное КПДП.                |
| 000Ch        | 0000h      | 3333h     | Третье слово, переданное КПДП.                |
| 0020h        | 8001h      | 8001h     | Исходное отрицательное число для команды SRA. |

|       |       |       |                                                                 |
|-------|-------|-------|-----------------------------------------------------------------|
| 0021h | 00F0h | 00F0h | Первый логический операнд.                                      |
| 0022h | 0F0Fh | 0F0Fh | Второй логический операнд для OR.                               |
| 0023h | 0000h | 0000h | Операнд для NOR над содержимым R2.                              |
| 0024h | 1234h | 1234h | Контрольное значение; из-за переходов не должно использоваться. |
| 0025h | FFFFh | FFFFh | Операнд для получения нулевого результата в R4.                 |
| 0030h | 0000h | C001h | Результат, записанный из регистра R3 командой MOV R3, 0030h.    |

В полном testbench внешний запрос DREQ удобно подавать после завершения команды по адресу 0010h, то есть сразу после записи C001h в память по адресу 0030h. При таком сценарии контроллер DMA успевает передать блок 1111h, 2222h, 3333h в ячейки 000Ah-000Ch ещё до того, как процессор выполнит последнюю команду MOV 000Ah, R7. Это позволяет на одной и той же временной диаграмме увидеть и результат арбитража шины, и непосредственное использование данных, записанных КППД.

### 3.2.2 Ожидаемые результаты сквозного выполнения программы

Контрольный расчёт по выбранной тестовой программе даёт следующую последовательность событий. После команд MOV 0020h, R1; SRA R1; INCS R1 регистр R1 принимает значение C001h. Команда PUSH помещает это слово в стек, а команда POP переносит его в регистр R3, после чего MOV R3, 0030h записывает C001h в ОЗУ. Пара команд MOV 0025h, R4 и NOR R4, 0025h формирует в регистре R4 нулевой результат, поэтому флаг Z становится равным единице и условный переход JZ по адресу 0016h обязан быть выполнен.

Поскольку условный переход выполняется, команда по адресу 0018h не должна попасть в фазу исполнения. Далее безусловный переход JMP переводит управление на адрес 001Eh, поэтому и команда по адресу 001Ch также пропускается. Если все счётчики предсказателя после сброса содержат код 01, то первое выполнение JZ обычно будет предсказано как «не переход». После разрешения реального условия в таблице РНТ для индекса PC(2)||GHR(2) = 1000b соответствующий счётчик должен измениться до 10, а регистр GHR - принять значение 01.

Последняя активная команда программы считывает по адресу 000Ah слово, записанное контроллером DMA, и помещает его в регистр R7. Таким образом, в финальном состоянии одновременно подтверждаются сразу несколько механизмов: корректная работа АЛУ и флагов, правильность LIFO-логики стека, работоспособность записи результата в ОЗУ, выполнение условного и безусловного переходов, а также возможность изменения памяти данных со стороны КППД без участия исполнительного тракта процессора.

**Таблица 16 - Контрольное финальное состояние модели**

| <b>Объект</b> | <b>Ожидаемое значение</b>     | <b>Комментарий</b>                                                               |
|---------------|-------------------------------|----------------------------------------------------------------------------------|
| R1            | C001h                         | Результат последовательности MOV -> SRA -> INCS.                                 |
| R2            | F000h                         | Результат логических операций OR и NOR.                                          |
| R3            | C001h                         | Слово, извлечённое из стека.                                                     |
| R4            | 0000h                         | После NOR R4, 0025h формируется нулевой результат.                               |
| R5            | 0000h                         | Не изменяется, так как команда по адресу 0018h пропущена.                        |
| R6            | 0000h                         | Не изменяется, так как команда по адресу 001Ch пропущена.                        |
| R7            | 1111h                         | Считывание первого слова, переданного КППД.                                      |
| R0, R8-R11    | 0000h                         | Эти регистры в программе не используются.                                        |
| FR            | Z = 1, S = 0,<br>C = 0, O = 0 | Флаги сохраняются после последней команды MOV.                                   |
| SP            | 7                             | После последовательности PUSH/POP стек логически пуст.                           |
| STK[6]        | C001h                         | Физически значение может оставаться в ячейке, но при SP = 7 оно уже не считается |

|        |    |                                                                        |
|--------|----|------------------------------------------------------------------------|
|        |    | вершиной стека.                                                        |
| GHR    | 01 | Результат обновления после одного выполненного условного перехода.     |
| PNT[8] | 10 | Счётчик для индекса 1000b после первого реально выполненного перехода. |

Фиксируя указанные значения на выходе модели после HLT, удобно быстро убедиться, что ключевые узлы работают согласованно. В частности, если R5 или R6 окажутся отличными от нуля, это будет означать ошибку в реализации переходов; если по адресу 0030h не появится C001h, значит нарушена работа тракта записи в память; если же R7 не примет значение 1111h, следует проверять либо КППД, либо логику арбитража шины данных.

### 3.2.3 Перечень сигналов для временных диаграмм

Чтобы временные диаграммы оставались читаемыми, не следует выводить на один лист абсолютно все внутренние сигналы проекта. Достаточно выбрать контрольные группы, по которым можно восстановить ход исполнения команды, обращения к памяти и работу служебных подсистем. Рекомендуемый состав сигналов для отчёта приведён в таблице 17.

**Таблица 17 - Рекомендуемые сигналы для временных диаграмм**

| Группа             | Сигналы                                                             | Что подтверждается                                                                              |
|--------------------|---------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| Глобальные сигналы | CLK, RESET, STATE[2:0], Tw                                          | Общая фазовая организация цикла команды, наличие или отсутствие состояний ожидания.             |
| Тракт команд       | IP[15:0], A_ROM[15:0], DO_ROM[15:0], IR0[15:0], IR1[15:0], IP_SEL   | Правильность выборки двух слов команды, загрузки регистров IR и смены адреса следующей команды. |
| РОН и АЛУ          | RF_WE, RF_WA, RF_DIN, A[15:0], B[15:0], ALU_OP, Y[15:0], Z, S, C, O | Корректность чтения и записи регистров, выбора операции АЛУ и обновления регистра               |

|                                    |                                                                                         |                                                                                              |
|------------------------------------|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
|                                    |                                                                                         | флагов.                                                                                      |
| Память данных и кэш                | A_RAM[15:0], RD_RAM, WR_RAM, DI_RAM[15:0], DO_RAM[15:0], CACHE_HIT, CACHE_MISS, LINE_WE | Попадания и промахи кэша, а также правильность циклов чтения и записи в ОЗУ.                 |
| Стек                               | SP, SP_DEC, SP_INC, STK_WE, STK_OE, STK_DIN[15:0], STK_DOUT[15:0]                       | Последовательность операций PUSH и POP, изменение указателя вершины и передача данных.       |
| Предсказатель переходов            | PC[1:0], GHR[1:0], PHT_IDX[3:0], PHT_STATE[1:0], PRED_TAKEN, BR_RESOLVE                 | Правильность выбора индекса, выдачи прогноза и обновления таблицы PHT и регистра GHR.        |
| КПДП и арбитр                      | DREQ, REQ_CPU, REQ_DMA, GNT_CPU, GNT_DMA, CURR_ADDR, BYTE_CNT, WR_RAM, DONE             | Приоритет DMA на общей шине, пакетная передача трёх слов и завершение обмена.                |
| Контрольные регистры полной модели | R1, R2, R3, R4, R7, FR, SP                                                              | Итог выполнения тестовой программы без необходимости выводить на диаграмму все 12 регистров. |

Таким образом, третий раздел задаёт полный набор исходных данных для моделирования и заранее фиксирует контрольные результаты, по которым удобно проверять работоспособность проекта. При фактическом оформлении курсовой работы в этот текст останется добавить только скриншоты временных диаграмм и, при необходимости, вынести полный листинг программы и расширенные дампы памяти в приложения. Содержательная часть проверки - что именно нужно наблюдать на диаграммах и какие значения считать правильными - уже определена.

## 4 АНАЛИЗ И ОПТИМИЗАЦИЯ РАЗРАБОТАННОЙ МИКРО-ЭВМ

Разработанная микро-ЭВМ ориентирована прежде всего на функциональную корректность и наглядность аппаратной структуры. Поэтому в базовой реализации многие действия выполняются строго последовательно: двухсловная команда полностью выбирается из синхронного ПЗУ, затем декодируется, после чего последовательно выполняются чтение операндов, обращение к памяти данных и запись результата. Такой подход упрощает проектирование, однако оставляет заметный резерв по быстродействию.

Наибольшее влияние на среднюю длительность команды оказывают три фактора: необходимость чтения двух 16-разрядных слов команды, возможные промахи кэша данных и потери времени при условных переходах. Дополнительные задержки создаёт и конкуренция за шину данных между процессорной частью и КППД. Следовательно, оптимизация должна быть направлена не на усложнение самого АЛУ, а на уменьшение числа состояний ожидания и на повышение степени параллелизма между блоками.

### 4.1 Сокращение длительности выполнения отдельных команд

Наименьшую длительность имеют чисто регистровые операции SRA и INCS, поскольку для них не требуется обращение к ОЗУ данных. Более долгими оказываются стековые команды PUSH и POP, так как в их составе присутствует обновление указателя вершины и собственно обмен со стековой памятью. Максимальную же длительность имеют команды, связанные с внешней памятью данных: MOV adr, reg, MOV reg, adr, OR reg, adr и NOR reg, adr. Их время зависит не только от внутренней последовательности микродействий, но и от наличия попадания или промаха в кэше.

Для качественной оценки удобно использовать относительную длительность команд в тактах автомата управления. Эти оценки не являются результатом временного анализа ПЛИС, но позволяют сравнить различные классы команд между собой и понять, на какие места архитектуры следует воздействовать при оптимизации.

**Таблица 18 - Оценка длительности выполнения основных классов команд**

| Класс команд  | Типичный состав фаз | Оценка длительности | Основной источник задержки |
|---------------|---------------------|---------------------|----------------------------|
| HLT, SRA reg, | Выборка двух слов,  | 4 такта             | Последовательна            |

|                                        |                                                          |            |                                                    |
|----------------------------------------|----------------------------------------------------------|------------|----------------------------------------------------|
| INCS reg                               | декодирование, исполнение, запись                        |            | я выборка команды из ПЗУ                           |
| JMP adr, JZ adr при верном прогнозе    | Выборка, декодирование, загрузка нового IP               | 4 такта    | Чтение второго слова команды и обновление IP       |
| PUSH reg, POP reg                      | Выборка, декодирование, обмен со стеком, изменение SP    | 5 тактов   | Последовательность работы со стековой памятью и SP |
| MOV adr, reg; OR/NOR reg, adr при hit  | Выборка, декодирование, доступ к кэшу, запись результата | 5 тактов   | Чтение операнда через кэш данных                   |
| MOV adr, reg; OR/NOR reg, adr при miss | То же + загрузка строки из ОЗУ                           | 7 тактов   | Промах кэша и ожидание ОЗУ                         |
| MOV reg, adr                           | Выборка, декодирование, запись через кэш и ОЗУ           | 5-6 тактов | Сквозная запись в память данных                    |
| JZ adr при ошибочном прогнозе          | Выборка, частичное исполнение, очистка неверного пути    | 6 тактов   | Сброс ошибочно выбранной последовательности команд |

Сократить длительность команд можно несколькими относительно простыми мерами. Во-первых, целесообразно ввести буфер выборки команды из двух слов. Тогда считывание второго слова можно частично перекрыть с декодированием первого, а устройство управления будет получать уже сформированную пару IR0/IR1. Во-вторых, полезно вынести формирование адреса следующей команды IP+2 в отдельный сумматор, чтобы не занимать на этой операции ресурсы основного АЛУ. В-третьих, для команд записи в память данных можно применить небольшой буфер записи: процессор завершает команду после загрузки слова в буфер, а фактическая сквозная запись в ОЗУ выполняется параллельно при освобождении шины данных.

Дополнительный выигрыш даёт объединение обновления SP с обменом данными в командах PUSH и POP. Если в блоке стека предусмотреть режим преддекремента и постинкремента, то изменение указателя вершины будет происходить в том же цикле, что и чтение или запись слова. Для условных переходов главным способом сокращения времени остаётся улучшение качества предсказания: чем реже возникает ошибочный прогноз, тем меньше потери на очистку неправильной последовательности выборки.

#### 4.2 Конвейерное выполнение фаз последовательностей команд

Наиболее естественным направлением развития архитектуры является введение конвейера выполнения команд. Для данной микро-ЭВМ рационально использовать четырёхстадийную схему с буфером выборки: F - выборка двух слов очередной команды в регистры IR0/IR1; D - декодирование и чтение операндов; E - выполнение операции, формирование адреса и обращение к кэшу или стеку; W - запись результата и обновление флагов. После заполнения такого конвейера несколько соседних команд могут находиться на разных стадиях одновременно, что заметно повышает среднюю производительность.

Гарвардская архитектура хорошо подходит для конвейеризации, поскольку выборка команд из ПЗУ не конфликтует с обычными обращениями к данным. Это означает, что уже на стадии F можно продолжать чтение следующих слов программы, пока текущая команда на стадии E работает с кэшем или со стеком. Однако полностью бесконфликтным конвейер не будет: остаются структурные, информационные и управляющие зависимости.

**Таблица 19 - Возможные конфликты конвейера и способы их разрешения**

| Тип конфликта   | Причина                                                  | Проявление в проекте                                                  | Способ разрешения                                                  |
|-----------------|----------------------------------------------------------|-----------------------------------------------------------------------|--------------------------------------------------------------------|
| Структурный     | Одна шина данных для CPU и КПДП                          | Процессор и DMA одновременно хотят обратиться к ОЗУ                   | Арбитраж шины и временная остановка стадии E для процессора        |
| RAW-зависимость | Следующая команда читает регистр, который ещё не записан | После MOV adr, reg или POP reg результат появляется только к стадии W | Прямая передача результата W->E или однократная задержка конвейера |

|             |                                                 |                                                                  |                                                               |
|-------------|-------------------------------------------------|------------------------------------------------------------------|---------------------------------------------------------------|
| Управляющий | Неизвестен фактический исход условного перехода | Для JZ целевой IP подтверждается только после вычисления флага Z | Использование предсказателя A4 и очистка конвейера при ошибке |
| Промех кэша | Требуется дополнительное чтение строки из ОЗУ   | Команды MOV/OR/NOR задерживают стадию E на несколько тактов      | Локальная остановка конвейера до прихода слова из ОЗУ         |

Даже при наличии перечисленных конфликтов конвейер остаётся полезным. В идеальном случае после заполнения конвейера можно получать завершение одной команды за такт. В реальной системе средний выигрыш будет меньше из-за промахов кэша и условных переходов, однако даже частичное перекрытие фаз позволяет уменьшить среднее время выполнения программы примерно в полтора-два раза по сравнению с полностью последовательной реализацией. Наибольший эффект ожидается на последовательностях из регистровых и стековых команд, где зависимость от внешней памяти минимальна.

Важно, что конвейеризация должна вводиться не вместо существующей модели, а поверх уже отлаженной архитектуры. Сначала подтверждается корректность однокомандного режима, затем выделяются границы стадий, вводятся регистры конвейера и только после этого анализируются конфликты. Такой порядок позволяет не потерять управляемость проекта и заметно упрощает отладку временных диаграмм.

### 4.3 Реализация микропрограммного устройства управления

В базовом проекте устройство управления рассматривается как жёстко заданный автомат. Для небольшого набора команд это решение эффективно по быстродействию, но по мере роста системы команд и числа специальных режимов оно становится менее удобным. Любое добавление новой команды требует переработки логики автомата, а анализ управляющих последовательностей усложняется. Альтернативой является микропрограммное УУ, в котором последовательность микродействий хранится в отдельной памяти управления.

В микропрограммном УУ каждая машинная команда при декодировании выбирает начальный адрес микропрограммы. Далее микросеквенсор последовательно извлекает микрокоманды из ПЗУ

управления, а их поля напрямую формируют управляющие сигналы для РОН, АЛУ, стека, кэша, памяти и указателя команд. Условные переходы внутри микропрограммы выполняются по состоянию флагов, признакам готовности памяти и сигналам арбитра. Такой подход позволяет явно описать последовательности для MOV, PUSH/POP, DMA-обмена и обновления предсказателя переходов.

**Таблица 20 - Пример формата микрокоманды микропрограммного уу**

| Поле                      | Разрядность | Назначение                                                    |
|---------------------------|-------------|---------------------------------------------------------------|
| SRC_A, SRC_B              | 2 x 4       | Выбор источников операндов для внутренних шин и АЛУ           |
| ALU_OP                    | 3           | Код операции АЛУ: OR, NOR, SRA, INCS, PASS и др.              |
| DST_SEL                   | 4           | Выбор регистра-приёмника или специального регистра назначения |
| REG_WE, FLAG_WE           | 2 x 1       | Разрешение записи в РОН и регистр флагов                      |
| MEM_RD, MEM_WR, CACHE_CTL | 1 + 1 + 2   | Управление доступом к ОЗУ/кэшу и режимом обслуживания строки  |
| STACK_CTL                 | 2           | Операции со стеком: нет действия, PUSH, POP, коррекция SP     |
| IP_CTL, IR_WE             | 2 + 1       | Инкремент IP, загрузка IP, разрешение загрузки IR             |
| NEXT_CTL, NEXT_ADDR       | 3 + 7       | Условие выбора следующей микрокоманды и её адрес              |

Для данного проекта объёма памяти управления порядка 64-128 микрокоманд достаточно с большим запасом. Например, команда MOV adr, reg может раскладываться на микродействия «выборка второго слова», «формирование адреса», «чтение через кэш», «запись в РОН», а команда JZ

adr - на микродействия проверки флага, выдачи прогноза и окончательного подтверждения нового IP. Расширение набора команд при таком подходе сводится главным образом к добавлению новых микропрограмм, а не к радикальной переделке логической схемы автомата.

Недостатком микропрограммного УУ является уменьшение быстродействия по сравнению с жёсткой логикой, поскольку на каждый шаг требуется доступ к памяти управления. Поэтому для текущего компактного технического задания жёсткая логика остаётся более рациональной. Однако микропрограммная реализация полезна как направление дальнейшего развития: она облегчает модернизацию системы команд, делает управление более прозрачным и позволяет включать новые сервисные режимы без полной переработки аппаратуры.

## ЗАКЛЮЧЕНИЕ

В ходе работы была разработана структура микро-ЭВМ на ПЛИС в соответствии с индивидуальным техническим заданием. Для проектируемой системы зафиксированы Гарвардская архитектура, 16-разрядные шины адреса и данных, синхронные ПЗУ и ОЗУ, 12 регистров общего назначения, стек глубиной 7 ячеек, кэш-память данных, блок предсказания переходов, контроллер прямого доступа к памяти и централизованный параллельный арбитр шины.

Сформирована система команд, включающая обязательные операции обмена с памятью, переходов и работы со стеком, а также заданные операции INCS, OR, NOR и SRA. Для всех основных блоков описаны назначение, структура, сигналы и взаимодействие при выполнении программы. Дополнительно подготовлена тестовая программа для сквозного функционального моделирования, контрольные дампы памяти и перечень сигналов, подлежащих выводу на временные диаграммы.

Проведённый анализ показал, что дальнейшее улучшение микро-ЭВМ связано прежде всего с уменьшением задержек памяти, с конвейеризацией выполнения команд и с возможным переходом к микропрограммному устройству управления. Следовательно, разработанная архитектура может рассматриваться как функционально завершённая базовая модель, пригодная как для моделирования, так и для дальнейшего усложнения при последующих этапах проектирования.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Содержание заданий курсового проекта по дисциплине «Структурная и функциональная организация ЭВМ». - Учебно-методический материал кафедры ЭВМ, 2011.
2. 1.BO6.DOC - методические материалы по организации базовых операций и функциональных узлов микро-ЭВМ.
3. 2.ALU6.DOC - методические материалы по построению арифметико-логического устройства.
4. 3.CUU6.DOC - методические материалы по построению централизованного устройства управления.
- 5..MPUU6.DOC - методические материалы по микропрограммному устройству управления.
6. 5.ADDRESS.DOC - методические материалы по режимам адресации и организации обращения к памяти.
7. Таненбаум Э. Архитектура компьютера. - СПб.: Питер.
8. Хеннесси Дж., Паттерсон Д. Архитектура компьютера и проектирование компьютерных систем. - СПб.: Питер.
9. Воронов Н. А. Образец пояснительной записки к курсовому проекту по разработке микро-ЭВМ на ПЛИС. - Учебный пример кафедры ЭВМ.

## ПРИЛОЖЕНИЯ

| Обозначение  | Содержание                                                                 |
|--------------|----------------------------------------------------------------------------|
| Приложение А | Графическая часть: структурные и функциональные схемы микро-ЭВМ.           |
| Приложение Б | Листинг тестовой программы, дампы памяти и контрольные данные.             |
| Приложение В | VHDL-листинги основных модулей проекта.                                    |
| Приложение Г | Материалы функционального моделирования и контрольные временные диаграммы. |
| Приложение Д | Инструкция по снятию временных диаграмм в Quartus/ModelSim.                |